Validating Formal Specifications using Testing-Based Specification Animation*

Shaoying Liu
Department of Computer Science
Hosei University, Japan
Email: sliu@hosei.ac.jp

ABSTRACT

Software requirements analysis and design can significantly benefit from writing formal specifications in some circumstances but meanwhile face challenges in validating the specifications. In this paper, we propose a specification animation technique to deal with the challenging issues. The technique is characterized by utilizing test case generation from formal operation specifications to carry out visualized demonstration of the input-output relation of the operation. With the animation technique, the analyst and the client can be facilitated to check the desirability of the specification. We describe a set of test case generation criteria based on both the specification structure and the client's requirements, and discuss how visualized demonstration can be implemented. We also present a small experiment to evaluate the effectiveness of our technique and briefly explain the potential challenges for future research.

1. INTRODUCTION

Despite the controversy over the feasibility of deploying formal methods in industry [1], quite a few industrial applications have demonstrated the benefits of using formal specifications in requirements analysis or design [2]. Our own experience with the development and application of the Structured Object-Oriented Formal Language (SOFL) [3, 4] has also indicated that if used properly together with existing software engineering technologies, formal specification can help software projects achieve good quality documentation and understanding of envisaged systems. However, this does not necessarily imply that formal specification techniques can easily fit into practical software projects. There are many challenges as described in the literature [5], but one of the major challenges, which we are interested in addressing in this paper, is the difficulty in validating specifications.

Specification animation has been proposed as a more practical approach than formal proof for specification verification and validation [6][7]. It is likely to facilitate the communication between the client and the developer due to its capability of allowing the specified behaviors to be demonstrated dynamically. It is also usually easier to adopt because it uses the idea of program testing familiar to practitioners at large. However, since the existing animation approaches require automatic translation from formal specifications into code for execution, their application is quite restricted because model-based formal specifications (e.g., specifications written using VDM, Z, B-Method) are generally nonexecutable. To overcome this hurdle, we have proposed the idea of utilizing test case generation from the pre- and post-conditions of an operation for testing specifications [8], which requires no translation from specifications to code, but the idea still remains undeveloped so far, in particular the unique feature of *multiple ports* of a *process* in the SOFL specification language has not been taken into account in test case generation.

In this paper, we describe the further development of our preliminary idea on specification animation proposed before into a more mature and usable level. Our new contributions are threefold. Firstly, a set of test case generation criteria is defined to meet the requirement for validating both the functional definition and the related constraints of a process in SOFL. Secondly, appropriate visualized demonstration of the input-output relation defined by the pre- and post-condition of a process is designed and supported by a tool. Finally, a small controlled experiment is conducted to investigate the effectiveness of our animation technique, see Section 6 for details.

Note that SOFL is used only as a "vehicle" language for the running examples and discussions in this paper; the underlying principle of our technique can be easily extended to other model-based formal notations because the structure of a process and its specification in SOFL is sufficiently general to cover the features of other similar model-based formal notations (e.g., VDM-SL, Z, B-Method, see Section 2.1 for a more detailed discussion). For the sake of space, we omit the summary of all sections in the paper.

2. OVERALL IDEA OF SPECIFICATION ANIMATION

We focus on specification animation for a single process in SOFL because this is a fundamental but challenging issue for formal specification animation. We first briefly introduce SOFL process specification before discussing issues on its animation.

*This work was supported by JSPS KAKENHI Grant Number 26240008.

FormaliSE'16, May 15 2016, Austin, TX, USA

DOI: http://dx.doi.org/10.1145/2897667.2897668

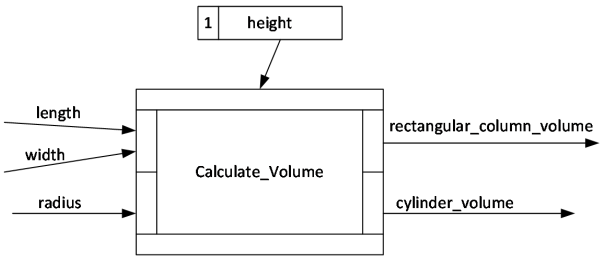

Figure 1: A process for calculating the volume of a rectangular column or cylinder

2.1 Process Specification

A process in SOFL models a transformation from input to output [4]. It is similar to an operation in VDM but has a more general structure to allow multiple input and output ports. This structure enables a process to be used more flexibly than the commonly used formal notations for abstraction in modeling systems, such as VDM-SL, Z, and B notation. This point will become clearer as the discussion on the structure of a process and its specification progresses.

A process is represented in both graphical and textual formats. For instance, Figure 1 shows a graphical representation of a process named *Calculate_Volume* for calculating the volume of a rectangular column or a cylinder, depending on what input data flows are provided. The process has two input ports, which are represented by two small narrow rectangles on the left side of the graphical representation. The upper input port receives two input data flows *length* and *width* representing the length and width of the base (the bottom level rectangle) of the rectangular column, while the lower input port receives only one input data flow *radius* representing the radius of the base (the bottom level circle) of the cylinder. It also has two output ports, which are represented by two narrow rectangles on the right side. The upper output port sends out a single output data flow *rectangular_column_volume* (denoting the volume of the corresponding rectangular column), while the lower output port sends out another output data flow *cylinder_volume* (denoting the volume of the corresponding cylinder). In addition, a data store named *height*, which indicates the height for either the rectangular column or the cylinder, is also connected to the process.

The graphical representation only specifies the functionality at abstract level. When both the input data flows *length* and *width* for the base of a rectangular column are available (for use), the volume of the rectangular column denoted by *rectangular_column_volume* will be produced. However, if *radius* for the base of a cylinder is available, the volume of the cylinder denoted by *cylinder_volume* will be calculated. In either case, the data store *height* is used properly in the calculation. This is also the reason why *height* is modeled as a store variable that is shared by both calculations.

One important feature of the process is that all of the input ports are *exclusive* in terms of the use of their input data flows. For example, if *length* and *width* are used to execute the process, the *radius* will not be used, and vice versa. The same rule also applies to output ports. This implies that either *rectangular_column_volume* or *cylinder_volume* is produced as the result of executing the process, but not both.

The detailed functionality of a process is formally specified using pre- and post-conditions in a textual format. For instance, a formal specification of the process *Calculate_Volume* is given as follows:

process *Calculate_Volume*(*length, width*: *real* | *radius: real*)
rectangular_column_volume: real |
cylinder_volume: real
ext rd *height*: *real*
pre *length >= 0 and width >= 0 and height >= 0*
or
radius >= 0 and height >= 0
post *bound(length, width) and*
rectangular_column_volume =
*length * width * height*
or
bound(radius) and
*cylinder_volume = radius * radius * 3.14 * height*
end_process;

In the signature of the process, the declarations of data flows of different input or output ports are separated using the vertical bar |. For instance, the declarations of the input data flows *length* and *width* are separated from that of the input data flow *radius*. The data store variable *height* is declared as an *external variable* (similar to a global variable) and only used as an input to the process, which is indicated by the keyword **rd** (read only) after **ext** (external). The pre-condition of the process requires that the related input variables and the initial external variable be greater than or equal to 0 when they are ready for use by the process. The post-condition defines *rectangular_column_volume* when both *length* and *width* are available (i.e., *bound(length, width)* is true) and *cylinder_volume* when *radius* is available.

On the basis of this example, we give a general definition of a process specification (simply a process) next.

DEFINITION 1. *A process is a six-tuple* (P, P_I, P_O, P_E, P_{pre}, P_{post})*, where* P *is the process name,* P_I *a collection of the input variable sets,* P_O *a collection of the output variable sets,* P_E *a set of the external variables including both decorated and undecorated external variables (if any),* P_{pre} *the pre-condition, and* P_{post} *the post-condition of the process.*

For instance, all of the elements of the process *Calculate_Volume* are as follows:

$Calculate\_Volume_I$ = {{*length*, *width*}, {*radius*}}
$Calculate\_Volume_O$ = {{*rectangular_column_volume*}, {*cylinder_volume*}}
$Calculate\_Volume_E$ = {*height*}
$Calculate\_Volume_{pre}$ = the same as given above
$Calculate\_Volume_{post}$ = the same as given above

Moreover, we also need a function that tells the type of a given variable in a formal specification to facilitate the discussion about test case generation later in this paper.

DEFINITION 2. *Let* $Variables$ *denote the universal set containing all possible variables and* $Types$ *the universal set containing all possible types in the SOFL specification language. Then, we define a function varType as follows:*
$varType : Variables \rightarrow Types$

Suppose $x \in A$ (with $A \in P_I$) is an input variable of process P, $varType(x)$ will denote the type of variable x. Applying the function to the two variables *length* and *radius* in

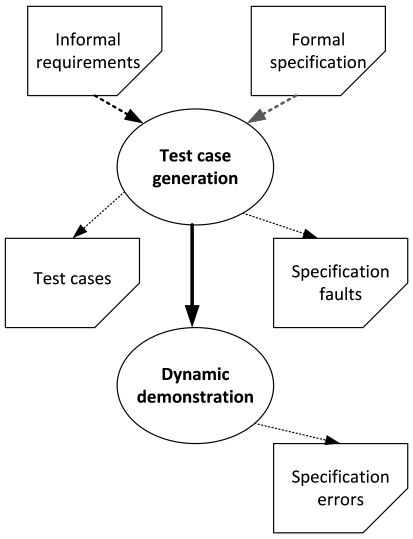

Figure 2: The procedure of a process specification animation

the specification of the process *Calculate_Volume* above, for example, we get $varType(length) = real$ and $varType(radius) = real$.

DEFINITION 3. *Let F be a set of predicate formulas (or expressions). Then, we define a function varSet as follows:* $varSet : F \rightarrow 2^{Variables}$.

where $2^{Variables}$ denotes the power set of the variable set $Variables$. When applied to a predicate formula Q, the function $varSet(Q)$ will yield the set of all the free variables occurring in Q. For example, $varSet(length >= 0$ *and* $width >= 0$ *and* $height >= 0) = \{length, width, height\}$.

2.2 Process Specification Animation

As mentioned previously, a process specification animation aims to check and to confirm whether the specification is desirable through dynamic demonstrations of the input-output relation using selected test cases. The procedure of carrying out an animation usually takes two steps, as illustrated in Figure 2. The first step is *test case generation* and the second step is *dynamic demonstration*.

Test case generation is aimed at selecting representative values for all of the variables involved in the specification, including input variables, output variables, and external variables. This point is similar to test case generation in program testing where both the input values and expected output values are usually prepared in advance, but the difference is that in the program testing, the expected output values are not used for executing the program while the output values generated for specification animation must be used during the animation, see more detailed discussions in Section 5.

Test cases can be generated based on either the formal specification or the informal requirements. The generation from the formal specification is similar to white-box testing for programs; it must take the structure of the specification into account in order to achieve desirable functional coverage for validation and to uncover potential faults in relation to specification consistency, see Section 3 for details. The generation based on the informal requirements, which may be documented in natural language, is similar to black-box testing for programs; it must take the user's requirements into account in order to ensure that the specified behaviors are expected, see Section 4. In either case, the activity of the test case generation may lead to the identification of faults because the process of generating test cases inevitably "force" the generator (either human or machine) to scrutinize either the formal specification or the informal requirements. That is, *test case generation can play the role of inspecting the specification and the requirements* in addition to preparing data for dynamic demonstration.

Dynamic demonstration for specification animation aims to demonstrate the input-output relation of a process in a visualized fashion in order to facilitate both the analyst (who wrote the specification) and the client to check whether the potential behaviors of the process are desirable.

3. TEST CASE GENERATION FROM SPECIFICATIONS

We first discuss the structure of a process specification and then proceed to describe the criteria for test case generation based on the structure.

3.1 Functional Scenarios

Our discussion focuses on process P. To keep the discussion concise, we assume that the related invariant, say inv, on state variables (i.e. external variables) has been incorporated into S_{pre} and S_{post} properly (as conjunction of inv and the original pre-condition or post-condition).

DEFINITION 4. *Let $P_{pre} = P_1 \vee P_2 \vee \cdots \vee P_n$ and $P_{post} = Q_1 \vee Q_2 \vee \cdots \vee Q_m$ be both a disjunctive normal form. Then, we call a conjunction $P_i \wedge Q_j$ $(i = 1, ..., n;\ j = 1, ..., m)$ a* functional scenario.

We treat the conjunction $P_i \wedge Q_j$ as a functional scenario because it defines an independent function: when P_i is satisfied by the input variables and the initial external variables, the output variables and the final external variables will be defined by Q_j.

DEFINITION 5. *Let $P_i \wedge Q_j$ be a functional scenario of process P. Then, $P_i \wedge Q_j$ is said* valid *if and only if the following condition holds:*
$\forall_{in1,in2 \in P_I} \cdot in1 \neq in2 \wedge varSet(P_i) \cap in1 \neq \emptyset \Rightarrow varSet(Q_j) \cap in2 = \emptyset$

A valid functional scenario $P_i \wedge Q_j$ is expected to ensure that the input satisfying P_i can be used in Q_j to define the output of the process, and therefore requires that P_i and Q_j do not contain input variables of different input ports due to the exclusiveness of input variables of the different input ports in executing process P.

Let us take the process *Calculate_Volume* as an example to illustrate the notions defined above. The relevant parts of the process are given as follows:

$Calculate\_Volume_{pre}$ = $P_1 \vee P_2$

$Calculate\_Volume_{post}$ = $Q_1 \vee Q_2$

where

P_1 = $length >= 0 \wedge width >= 0 \wedge height >= 0$

P_2 = $radius >= 0 \wedge height >= 0$

Q_1 = $bound(length, width) \wedge rectangular\_column\_volume = length * width * height$

Q_2 = $bound(radius) \wedge cylinder\_volume = radius * radius * 3.14 * height$

Functional scenarios : (1) $P_1 \wedge Q_1$; (2) $P_1 \wedge Q_2$; (3) $P_2 \wedge Q_1$; (4) $P_2 \wedge Q_2$

Valid functional scenarios : (1) $P_1 \wedge Q_1$; (2) $P_2 \wedge Q_2$

DEFINITION 6. *We use P_{FS} to denote the set of all possible functional scenarios of process P and P_{VFS} the set of all possible valid functional scenarios of process P.*

These two symbols will be used later in this paper.

3.2 Test Case Generation Criteria

Before discussing the criteria for test case generation, we first need to clarify the basic concepts *test case* and *test set* because they are used in our work with slightly different meaning from the conventional use in program testing.

DEFINITION 7. *Let P be a process. Let $P_I = \{X_1, X_2, ..., X_n\}$, $P_O = \{Y_1, Y_2, ..., Y_m\}$, $P_E = \{\tilde{}z_1, \tilde{}z_2, ..., \tilde{}z_w, z_1, z_2, ..., z_w\}$. Then, a* test case *for P, denoted by tc, is a mapping from P_V to the set $Values$:*

$tc : P_V \rightarrow Values$;

$tc(v) \in varType(v)$

where $P_V = \bigcup_{i \in P_I} X_i \cup \bigcup_{i \in P_O} Y_i \cup P_E$

and $Values = \bigcup_{v \in P_V} varType(v)$.

Note that this definition only explains what a test case means; it is not used as a test case generation criterion for our purpose. Test criteria will be presented later in this section.

A test case is usually expressed as a set of pairs of variable with its value, for example, $tc = \{(x_1, 5), ..., (\tilde{}z_1, 10),$ $..., (y_1, 50), ..., (z_1, 20), ...\}$ is a possible test case for process P, , where $\tilde{}z_1$ denotes the initial external variable of a **wr** (writable) type external variable z.

To facilitate our discussions in the rest of this paper, we divide a test case tc into two parts: (1) input test case tc_i, which contains values only for the input variables and the initial external variables (e.g., $tc_i = \{(x_1, 5), ..., (x_n, 70),$ $(\tilde{}z_1, 10), ..., (\tilde{}z_w, 80)\}$), and (2) output test case tc_o, which contains values only for the output variables and the final external variables (e.g., $tc_o = \{(y_1, 50), ..., (y_m, 210),$ $(z_1, 20), ..., (z_w, 380)\}$). Such a distinction between input and output test cases will allow us to easily describe the input-output relation for a process in a dynamic demonstration, see details of Section 5.

For example, a possible test case for process *Calculate_Volume* is:

tc = $\{(length, 10), (width, 5), (radius, 8),$ $(rectangular\_column\_volume, 300),$ $(cylinder\_volume, 1205.76), (height, 6)\}$,

tc_i = $\{(length, 10), (width, 5), (radius, 8),$ $(height, 6)\}$,

tc_o = $\{(rectangular\_column\_volume, 300),$ $(cylinder\_volume, 1205.76), (height, 6)\}$.

DEFINITION 8. *A test set for process P is a set of test cases for process P.*

We can now define a set of criteria for generating a test set from a process specification, each specifying a standard for measuring whether the generated test set is adequate for a test.

Criterion 1: Let T be a test set generated from the specification of process P. Then, T must satisfy the following conditions:

(1) $\forall_{Ip \in P_I} \exists_{tc \in T} \cdot ((\forall_{iv \in Ip} \cdot tc(iv) \in varType(iv)) \Rightarrow \exists_{f \in P_{VFS}} \cdot f(tc))$

(2) $\forall_{Op \in P_O} \exists_{tc \in T} \cdot ((\forall_{ov \in Op} \cdot tc(ov) \in varType(ov)) \Rightarrow \exists_{f \in P_{VFS}} \cdot f(tc))$

(3) $\forall_{\tilde{}ev \in P_E} \forall_{ev \in P_E} \exists_{tc \in T} \cdot ((tc(\tilde{}ev) \in varType(ev) \wedge tc(ev) \in varType(ev)) \Rightarrow \exists_{f \in P_{VFS}} \cdot f(tc))$

where $tc(iv)$ denotes the value bound for variable iv in test case tc and $f(tc)$ means that test case tc satisfies the functional scenario f.

This criterion is the least but essential requirement for generating adequate test set T. Intuitively, it requires that for all of the input variables of every input port and output port, a test case that satisfies some valid functional scenario must be generated for animation. It also requires that for all initial and final external variables a test case satisfying some valid functional scenario must be generated. Thus, it ensures that all of the variable groups involved in the process specification are tested at least once, respectively.

Due to the fact that the input ports or output ports do not have a one-to-one relation with the valid functional scenarios derived from the pre- and post-conditions, this criterion apparently does not ensure that every valid functional scenario is tested. For this reason, we need another criterion.

Criterion 2: Let T be a test set generated from the specification of process P. Then, T must satisfy the following condition:

$\forall_{f \in P_{VFS}} \exists_{tc_1 \in T} \cdot f(tc_1)$

Intuitively, this criterion requires that every valid functional scenario be dynamically demonstrated with at least one test case in test set T. This is reasonable because every valid functional scenario is supposed to define a meaningful and hopefully desirable behavior of the process and therefore its validity must be checked.

However, due to the possibility of human mistakes, some of the invalid functional scenarios of the process might be either wrongly defined (e.g., an invalid scenario should be a valid one) or improperly defined (e.g., using wrong terms or operators defined on types). For this reason, we define another criterion to allow every possible functional scenario to be tested and demonstrated in the animation.

Criterion 3: Let T be a test set generated from the specification of process P. Then, T must satisfy the following

condition:

$\forall_{f \in P_{FS}} \exists_{tc_1 \in T} \cdot f(tc_1)$

This criterion allows us to cover all of the functional scenarios in animation.

4. TEST CASE GENERATION FROM REQUIREMENTS

Another important aspect to cover in specification animation is to check whether the user's requirements are satisfied by the specification using adequate test cases. Intuitively, the underlying principle for generating test cases should be to ensure that every important functional and non-functional requirement is demonstrated, respectively, regardless of the specification structure. In our approach, we assume that the requirements for a process are written in a SOFL informal specification, which is composed of three sections: (1) functions (describing desired behaviors), (2) data resources (data items necessary for implementing the functions), and (3) constraints (either on the data resources or functions). Consider the process *Calculate_Volume* as an example. Its informal requirements specification is written as follows:

1. Functions

1.1 Calculate the volume of an object

1.1.1 Compute the volume of a rectangular column

1.1.2 Compute the volume of a cylinder

2. Data Resources

2.1 *height* for the rectangular column and the cylinder

2.2 *length* and *width* for the bottom rectangle of the rectangular column

2.3 *radius* for the bottom circle of the cylinder

3. Constraints

3.1 *length*, *width*, *height*, and *radius* cannot be negative real numbers.

3.2 *height* cannot be over $100,000$.

Abstractly, we use a triple (F, D, C) to represent such a requirement specification, where F is a set of functions, D a set of data resources, and C a set of constraints. We can then define another criterion for test case generation that takes the user's requirements into account.

Criterion 4: Let T be a test set generated from the requirements specification (F, D, C) of process P. Then, T must satisfy the following conditions:

(1) $\forall_{f \in F} \exists_{tc \in T} \cdot tested(f, tc)$

(2) $\forall_{d \in D} \exists_{tc \in T} \cdot tested(d, tc)$

(3) $\forall_{c \in C} \exists_{tc \in T} \cdot tested(c, tc)$

where $tested(f, tc)$, $tested(d, tc)$, and $tested(c, tc)$ mean that f, d, and c are tested with the test case tc, respectively.

This criterion expresses the necessity of generating a test case for each function, data resource, and constraint to demonstrate the related behavior of the process. Such a test checks whether every function, data resource, and constraint described in the informal specification is properly defined in the formal specification.

5. DYNAMIC DEMONSTRATION

The second step in specification animation is dynamic demonstration of the input-output relation. To enable the demonstration to effectively help humans (analyst or client or both) make judgements on the validity of the specification, we have developed a software tool to support the input-output relationship demonstration graphically. Since a detailed introduction is beyond the scope of this paper, we only present the necessary part related to the topic of this section. All of the other details of the tool are described in Li's doctoral thesis [9].

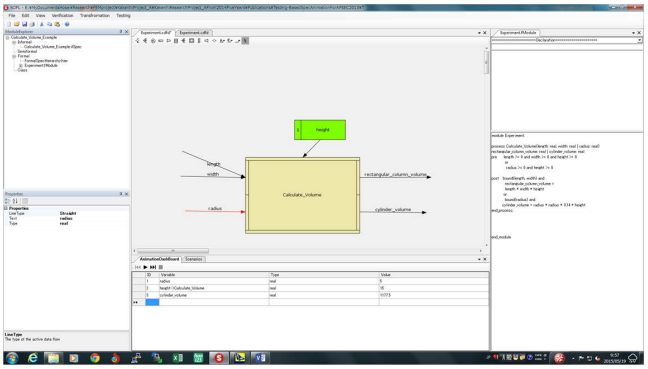

Figure 3: Snapshot of the dynamic demonstration for animation of process Calculate_Volume

Before demonstrating the behavior of a process, the tool facilitates the user to choose a functional scenario and allows test cases to be generated either automatically or manually. At the moment, the sub-tool for automatic test case generation does not function properly and its improvement is still undergoing. Once a test case is made available, by clicking on the right button, the tool will start to show how the selected input data flows are used to produce the expected output data flows. If data stores are connected to the process under animation, the access or updating of the stores is also properly demonstrated. To attract the user's attention, the tool may choose different colors to represent the input and output data flows, respectively.

Figure 3 shows a snapshot of dynamic demonstration of the specification of the process *Calculate_Volume*. The process is represented graphically in the middle pane of the GUI and its formal specification is given in the right pane. When the input data flow *radius* is selected, its test case can be generated and presented in the area below the pane of the graphical representation. The test case can be provided by the user. When the arrow button at the left-bottom of the middle pane is clicked, a dynamic demonstration of the behavior of producing *cylinder* from the input data flow *radius* will be performed comprehensibly.

6. SMALL EXPERIMENT

We have conducted a relatively small experiment, due to many resource constraints, on our testing-based specification animation approach in which test cases are generated manually. The goal is to observe (rather than rigorously evaluate) its effectiveness in terms of fault detection rate and comparing its performance with an existing *rigorous review approach*. The review approach is the *only other technique comparable with our approach in this case*; it requires the reviewer to read and examine each functional scenario formed using the same way as introduced previously. Other approaches, such as formal proof or existing specification animation methods, could be used for comparison with our method, but since formal proof is too difficult to carry out for our specifications and the existing animation methods can-

Processes	Injected faults	Detected faults by Group *A*		Detected faults by Group *B*	
		S1	*S2*	*S3*	*S4*
Register_Card	23	23	15	10	1
Charge_With_Cash	15	15	5	5	1
Charge_From_Bank	21	21	19	12	1
Buy_With_Card	15	14	5	7	1
Buy_With_Card_Cash	5	5	2	4	1
Entering_Station	24	19	11	10	2
Exiting_Station	34	34	17	23	2
Update_Commute_Ticket	19	17	19	14	4
Total	**156(100%)**	**148(95%)**	**93(60%)**	**85(54%)**	**13(8%)**

Figure 4: The experiment result

not be applied in our case because they require automatic transformation from the specifications into code that is infeasible for our mutant specifications, we could not adopt them. One may suggest that we should compare our approach with automatic test case generation approach, but due to many injected faults, our supporting tool for automatic test case generation does not work properly because without correcting those faults, appropriate test cases are impossible to produce automatically. This situation also applies to other existing test case generation tools [10]. The only applicable and comparable technique with ours is the review approach just mentioned above.

Part of the SOFL specification developed by our research group for the future *Suica card* (an Integrated Circuit card) system for the East Japan Railway Company is chosen as the target specification in the experiment. The system allows the user of the card to enjoy various railway services including *registering a card*, *buying tickets*, *charging card with cash* or *from bank account*, *entering station*, *exiting station*, and *updating commutation ticket*. The specification includes 6 modules, 8 processes, 41 data flows, 11 data stores, and approximately 315 lines of expressions.

Four senior students of our research group with the experience of using SOFL before were chosen to join the experiment. They were divided into two groups A and B with the assurance of equivalent capability between the two groups that is judged by their supervisor based on the students' academic performance. The two students in group A, who are named $S1$ and $S2$, used our animation method while the two students in group B, who are named $S3$ and $S4$, used the rigorous review approach to check the target specification. The target specification is actually a mutant specification resulted from injecting certain faults by another independent researcher in our lab. The fault were injected mainly by applying some mutation operators normally used for mutation testing, including operator replacement, insertion, and deletion in arithmetic expressions, relational conditions, and compound logical expressions used in pre- and post-conditions.

Figure 4 shows the overall experiment result in terms of detecting injected faults. The major points that can be derived from the result are summarized as follows:

- The fault detection rate of both students $S1$ and $S2$ using our animation method in group A is quite high; one is approximately 95% and the other is 60%.
- The test case generation by hand conducted in the experiment can help find approximately 95% of the total detected faults according to our detailed statics. This result mainly benefits from the unavoidable close examination of the details of every level expression during the formation of functional scenarios and test case generation. In group A, $S2$ found less faults than $S1$ in almost every case because $S2$ directly generated test cases based on the mutant specification while $S1$ first corrected obvious faults and then generated test cases. In group B, $S4$ performs much less effectively than $S3$ because $S4$ did not work as hard as $S3$, as far as we observed.
- On average, the two students in group A using our method finds approximately 46.5% more faults than the students in group B using the rigorous review method.

One important experience derived from this experiment is that manually generating test cases can be extremely effective in finding faults. Well-trained human developers (including testers) are an important "technology" in practice and their roles can hardly be completely replaced by automatic techniques. Since the scale of our experiment in this paper is still relatively small, its result can only be treated as preliminary and the validity of the result will have to be further investigated through larger scale empirical studies in the future.

7. RELATED WORK

As far as verification and validation of formal specifications are concerned, several approaches have been studied according to the literature. They include *formal verification*, *review/inspection*, and *testing/simulation/animation*. Since our work in this paper is closely related to the last category, our introduction to related work concentrates only on this category. Even this, we still cannot provide a comprehensive summary for the sake of space.

PiZA [6] is an animator for Z formal specification. It translates Z specifications into Prolog to generate outputs. Similarly, an animation approach for Object-Z specification is described in [11], which translates the Z specification into C++ code for execution. Morrey *et al.* developed a tool called "wiZe" to support the construction of model-based specification language, in particular Z, and the animation of an executable subset of Z specification language [12]. The subtool for animation of specifications is called ZAL. The wiZe is responsible for making a syntactically correct specification and transforming it into an executable representation in an extended LISP, and then passes the executable representation to ZAL. ZAL animates the specification by executing the specification with test cases. PVS (Prototype Verification System) is a specification and verification system including an expressive specification language and interactive theorem prover. It includes a ground evaluator [13] that can translate an executable subset of PVS to Common Lisp and provides a read-eval-print loop for testing the translated Lisp program. Combes and his colleagues described an open animation tool for telecommunication systems in [14]. Miller and Strooper introduced a framework for animating model-based specifications by using testgraphs [7]. The framework provides a testgraph editor for users to edit testgraphs, and then derive sequences for animation by traversing the testgraph. Gargantini and Riccobene proposed an automatically driven approach to animating formal specifications in

Parnas' SCR tabular notation [15]. One important feature of this work is the adoption of a model checker to help find counter-examples that contain a state not satisfying the property to be established by animation. VDMTools is an industry-strength toolset supporting the analysis of system models expressed in VDM [16]. The interpreter inside the VDMTools can execute a large executable subset of VDM specification. User can test the VDM specification by providing test cases and observe the system behavior by setting breakpoints or stepping. Overture [17] provides functions similar to VDMTools and is built on an open and extensible platform based on the Eclipse framework.

Compared to the existing studies, our method is characterized by the fact that *it neither requires automatic transformation from formal specifications to code nor needs to involve complicated formal verification techniques.* It is only based on the test case generation from logical expressions and visualized demonstration, therefore can be easily applied to all of the pre-post style formal notations in principle because an operation in Z or VDM, for example, is equivalent to a process in SOFL with a single input port and a single output port.

8. CONCLUSION AND FUTURE WORK

We have put forward a testing-based specification animation approach to validating formal specifications. The approach includes two steps: test case generation and visualized demonstration. We have also presented a set of criteria for generating test cases, explained the underlying principle for the visualized demonstration with a tool support, and described a small experiment on our method. The experiment has shown some positive aspects about our method compared with the rigorous review method, although its result is still limited considering the scale of the target specification and the subjects involved.

Our future work will concentrate on large scale empirical studies to evaluate the performance of our animation method. We will also extend the functionality of the current tool to support both manual and automatic test case generation and to integrate data animation into the input-output relation animation presented in this paper.

Acknowledgment: I would like to thank Mo Li for his considerable efforts in building the supporting tool for SOFL formal specification construction and animation.

9. REFERENCES

[1] D. L. Parnas, "Really Rethinking Formal Methods", *Computer*, vol. 43, no. 1, pp. 28–34, January 2010.

[2] J. Woodcock, P. G. Larsen, J. Bicarregui, and J. Fitzgerald, "Formal Methods: Practice and Experience", *ACM Computing Surveys*, vol. 41, no. 4, pp. 1–39, 2009.

[3] S. Liu, A.J. Offutt, C. Ho-Stuart, Y. Sun, and M. Ohba, "SOFL: a Formal Engineering Methodology for Industrial Applications", *IEEE Transactions on Software Engineering*, vol. 24, no. 1, pp. 337–344, January 1998, Special Issue on Formal Methods.

[4] S. Liu, *Formal Engineering for Industrial Software Development Using the SOFL Method*, Springer-Verlag, ISBN 3-540-20602-7, 2004.

[5] T. Leavens, K. Rustand, M. Leino, and P. Muller, "Specification and Verification Challenges for Sequential Object-Oriented Programs", *Formal Aspects of Computing*, , no. 19, pp. 159–189, 2007.

[6] M. Hewitt, C. O'Halloran, and C. Sennett, "Experiences with PiZA: an Animator for Z", in *ZUM'97*. 1997, vol. 1212 of *LNCS*, pp. 37–51, Springer.

[7] Tim Miller and Paul Strooper, "A Framework and Tool Support for the Systematic Testing of Model-Based Specifications", *ACM TOSEM*, vol. 12, no. 4, pp. 409–439, 2003.

[8] S. Liu, "Verifying Consistency and Validity of Formal Specifications by Testing", in *Proceedings of the World Congress on Formal Methods in the Development of Computing Systems*, Toulouse, Sept. 1999, LNCS, pp. 896–914, Springer.

[9] Mo Li, *A Systematic Inspection Approach to Verifying and Validating Formal Specifications based on Specification Animation and Traceability*, PhD thesis, Hosei University, Tokyo, Japan, July 2015.

[10] S. Khurshid and D. Marinov, "TestEra: Specification-based Testing of Java Programs using SAT", *Automated Software Engineering*, vol. 11, no. 4, pp. 403–434, 2004.

[11] M. Najafi and H. Haghighi, "An animation approach to develop c++ code from object-z specifications", in *2011 CSI International Symposium on Computer Science and Software Engineering (CSSE 2011)*. IEEE, 2011, pp. 9–16.

[12] I. Morrey, J. Siddiqi, R. Hibberd, and G. Buckberry, "A Toolset to Support the Construction and Animation of Formal Specifications", *Journal of Systems and Software*, vol. 41, no. 3, pp. 147–160, June 1998.

[13] J. Crow, S. Owre, J. Rushby, N. Shankar, and D. Stringer-Calvert, "Evaluating, testing, and animating pvs specifications", *Computer Science Laboratory, SRI International, Menlo Park, CA, Tech. Rep*, 2001.

[14] P. Combes, F. Dubois, and B. Renard, "An Open Animation Tool: Application to Telecommunication Systems", *Computer Networks: The International Journal of Computer and Telecommunications Networking*, vol. 40, no. 5, pp. 599–620, 2002.

[15] A. Gargantini and E. Riccobene, "Automatic Model Driven Animation of SCR Specifications", in *FASE 2003*, Mauro Pezzè, Ed., Warsaw, Poland, 2003, vol. 2621 of *LNCS*, Springer.

[16] J. Fitzgerald, P. G. Larsen, and S. Sahara, "Vdmtools: Advances in support for formal modeling in vdm", *ACM Sigplan Notices*, vol. 43, no. 2, pp. 3, 2008.

[17] P. G. Larsen, N. Battle, M. Ferreira, J. Fitzgerald, K. Lausdahl, and M. Verhoef, "The overture initiative integrating tools for vdm", *SIGSOFT Softw. Eng. Notes*, vol. 35, no. 1, pp. 1–6, Jan. 2010.